Maschinelles Lernen für die Chemie - Exercise 3

Logistic Regression, Support Vector Machines

Leonie Pfeffer, Maike Vogelbacher | May 23, 2025

Logistic Regression for Classification

Definition

- Supervised learning
- Predict categorical outcomes
  - Binary classification: yes/no, 1/0
  - Multiple classes

Goal

Predict the probability that a sample x belongs to class 1

$$P(y = 1 \mid x)$$

- $x \in \mathbb{R}$: a single input feature
- $y \in \{0, 1\}$: target class label

Decision Rule

Binary classification with decision boundary at 0.5

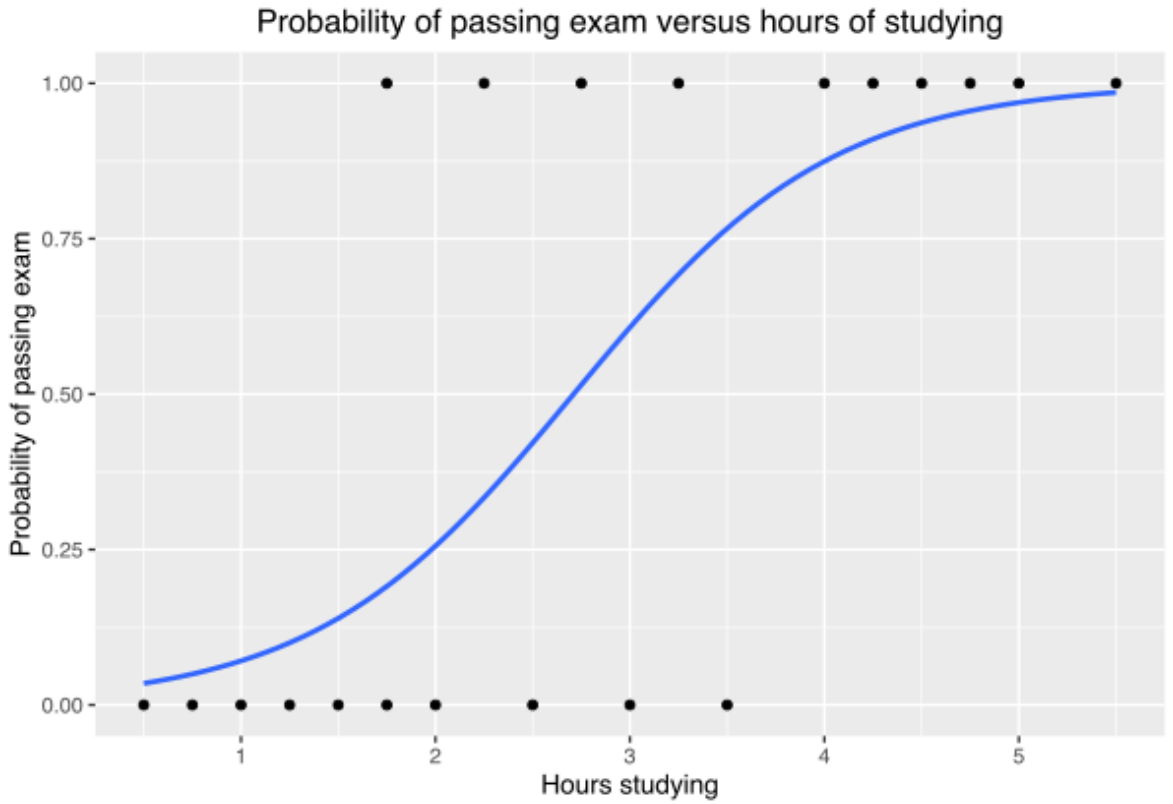
$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1 \mid x) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Sigmoid Function

a.k.a logistic function

$$P(y = 1 \mid x) = \frac{1}{1 + \exp[-(wx + b)]}$$

- w: weight (slope)
- b: bias (intercept)



Multi-class Classification: Multinomial Regression

Example: Iris Dataset

- $n = 150$ samples
- $d = 4$ features (sepal length/width, petal length/width)
- $K = 3$ classes (setosa, versicolor, virginica)
- $\boldsymbol{x}^{(i)} \in \mathbb{R}^4$ is the feature vector for the i^{th} sample
- $\boldsymbol{y}^{(i)} \in \{0, 1, 2\}$ is the class label for the i^{th} sample

Multinomial Regression (softmax)

Idea: learn a single model with one set of weights per class, all weights are learned simultaneously

$$P(y = k \mid \boldsymbol{x}) = \frac{\exp[-(\boldsymbol{w}_k^T \boldsymbol{x})]}{\sum_j \exp[-(\boldsymbol{w}_j^T \boldsymbol{x})]}$$

- Weights: $W \in \mathbb{R}^{3\times4}$, where row k is $\boldsymbol{w}_k^T$
- Bias: $\boldsymbol{b} \in \mathbb{R}^3$
- $i \in \{1, ..., 150\}$
- Result: single vector of three class probabilities, summing to 1

Multi-class Classification: One-vs-Rest

Example: Iris Dataset

- $n = 150$ samples
- $d = 4$ features (sepal length/width, petal length/width)
- $K = 3$ classes (setosa, versicolor, virginica)
- $\boldsymbol{x}^{(i)} \in \mathbb{R}^2$ is the feature vector for the i^{th} sample
- $\boldsymbol{x}^{(i)} \in \{0, 1, 2\}$ is the class label for the i^{th} sample

One-vs-Rest (sigmoid)

Idea: train K separate binary classifiers, one for each class k, to distinguish class k from all others

$$P(y = k \mid \boldsymbol{x}^{(i)}) = \frac{1}{1 + \exp[-(\boldsymbol{w}_k^T \boldsymbol{x}^{(i)} + b_k)]}$$

- Weights: $\boldsymbol{w}_k \in \mathbb{R}^4$
- Bias: $b_k \in \mathbb{R}$
- $i \in \{1, ..., 150\}$
- Result: three scalar probabilities per sample

Exercise: One-vs-Rest Regression

Training

Create new labels

$$y^{(i)} = \begin{cases} 1 & \text{if original class is } k \\ 0 & \text{otherwise} \end{cases}$$

Cross-entropy loss function

$$\mathcal{L}_k = -\sum_{i=1}^{n} \left[y_k^{(i)} \log(\hat{y}_k^{(i)}) + (1 - y_k^{(i)}) \log(1 - \hat{y}_k^{(i)}) \right]$$

- Typical for classification
- Can be optimized well due to global minimum

Prediction

Predictor for each class k

$$\hat{y}_k = \frac{1}{1 + \exp[-(\mathbf{w}_k^T \mathbf{x} + b_k)]}$$

Select classifier with highest probability

$$\hat{y} = \arg\max_k \hat{y}_k$$

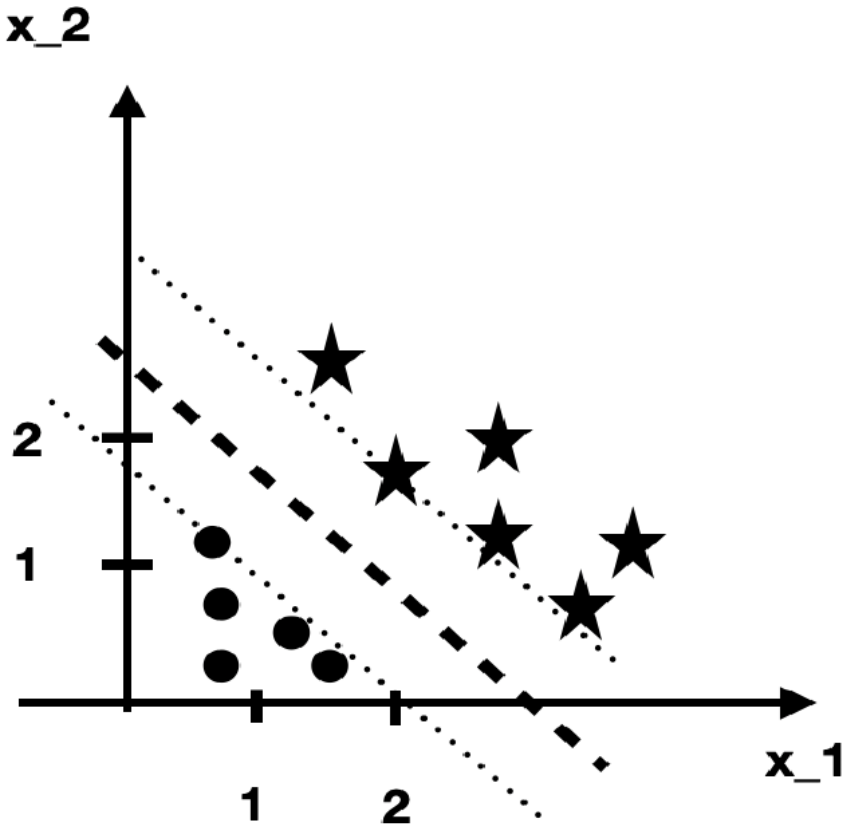
Support Vector Classifiers – Concepts

Support Vector Classifier (SVC)

- Linear classifier
- Best possible hyperplane that divides data
- Idea: find the maximum margin

Margin

- Distance between the hyperplane and the closest data points
- Closest points: support vectors
- Hard-margin: no misclassifications allowed
- Soft-margin: slack variables allowing misclassifications (regularization parameter)



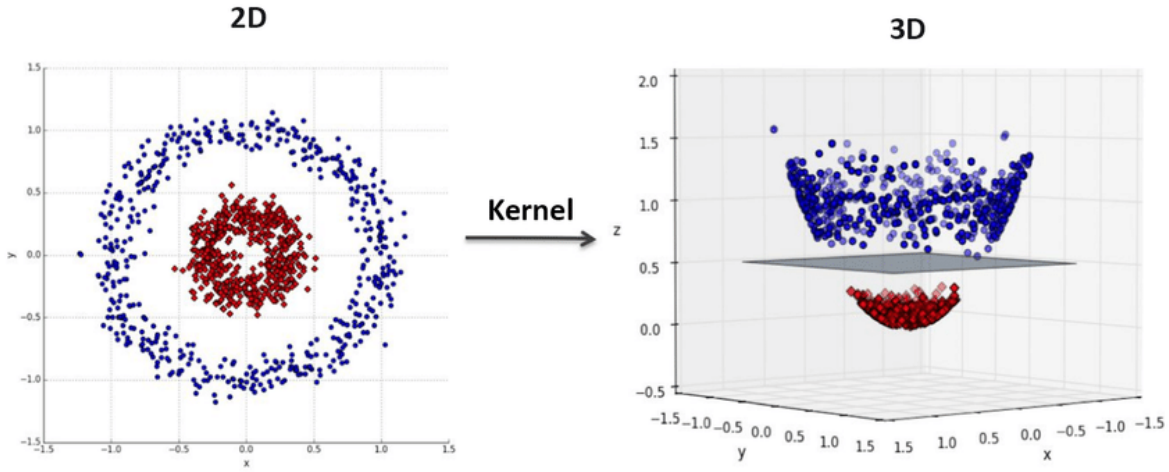
Support Vector Machines (SVMs)

Nonlinear Classifier

- Start with the low-dimensional data
- Transform the data to a higher dimension
- Use a linear SVC to separate the data

Kernel Trick

- Kernel trick: calculate the relationship between different points in a higher dimension **without** the actual transformation of the data points
- Kernel function: computes the inner dot product between two points on the higher-dimensional space

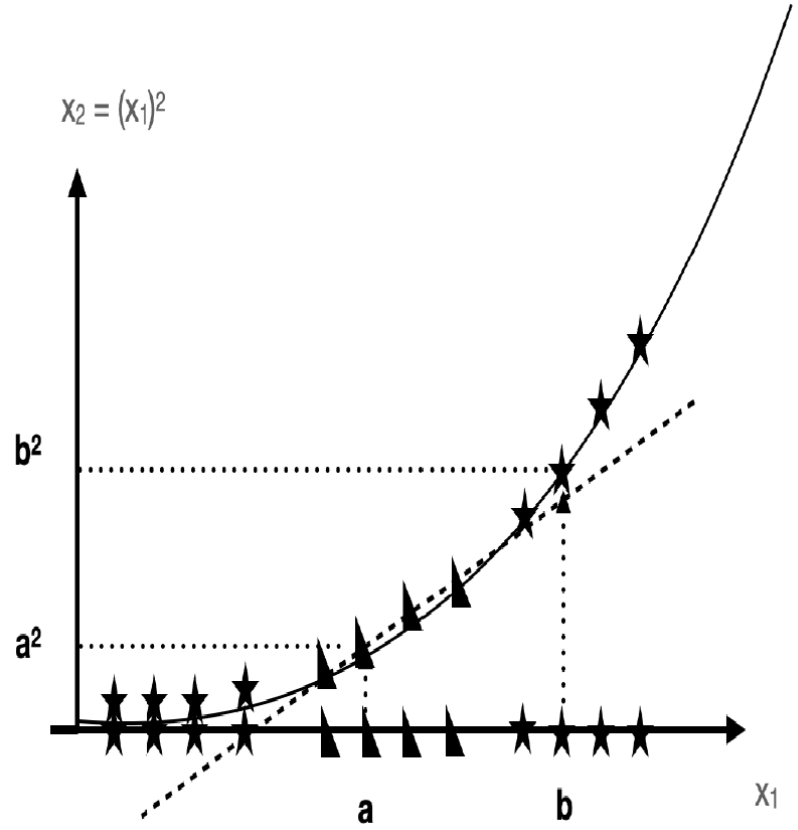


Polynomial Kernel

Equation

$$K(x, x') = (1 + \langle x, x' \rangle)^d$$

- Degree d (hyperparameter)
- Dot product between the two features x and x': $\langle x, x' \rangle$



Radial Basis Function (RBF)

Equation

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

- a. k. a. radial kernel or Gaussian kernel
- Squared Euclidean distance: $\|x - x'\| = \sum_i (x_i - x'_i)^2$
- Width of Gaussian: $\gamma > 0$ (how quickly does the similarity decay)

Interpretation

- Technically has infinite dimensions (exponential function can be expanded in a Taylor series of infinite terms)
- When $x = x'$ (identical data points), the kernel is 1 (maximum similarity):

$$K(x, x) = \exp(0) = 1$$

- As $\|x - x'\|$ increases, $K(x, x') \to 0$
- The smaller γ, the slower K decays (wider influence)
- The larger γ, the faster K decays (narrower influence)

Pandas Methods

`pandas.DataFrame.value_counts()`

- Count the number of occurrences of each unique value in a DataFrame

`pandas.DataFrame.info()`

- Print summary of the DataFrame
- Similar to `pandas.DataFrame.describe()`

`pandas.DataFrame.copy()`

- Copy selected sub-DataFrames (e. g. columns) to a new DataFrame variable
- Changes in the original will not influence the copy (default behavior)

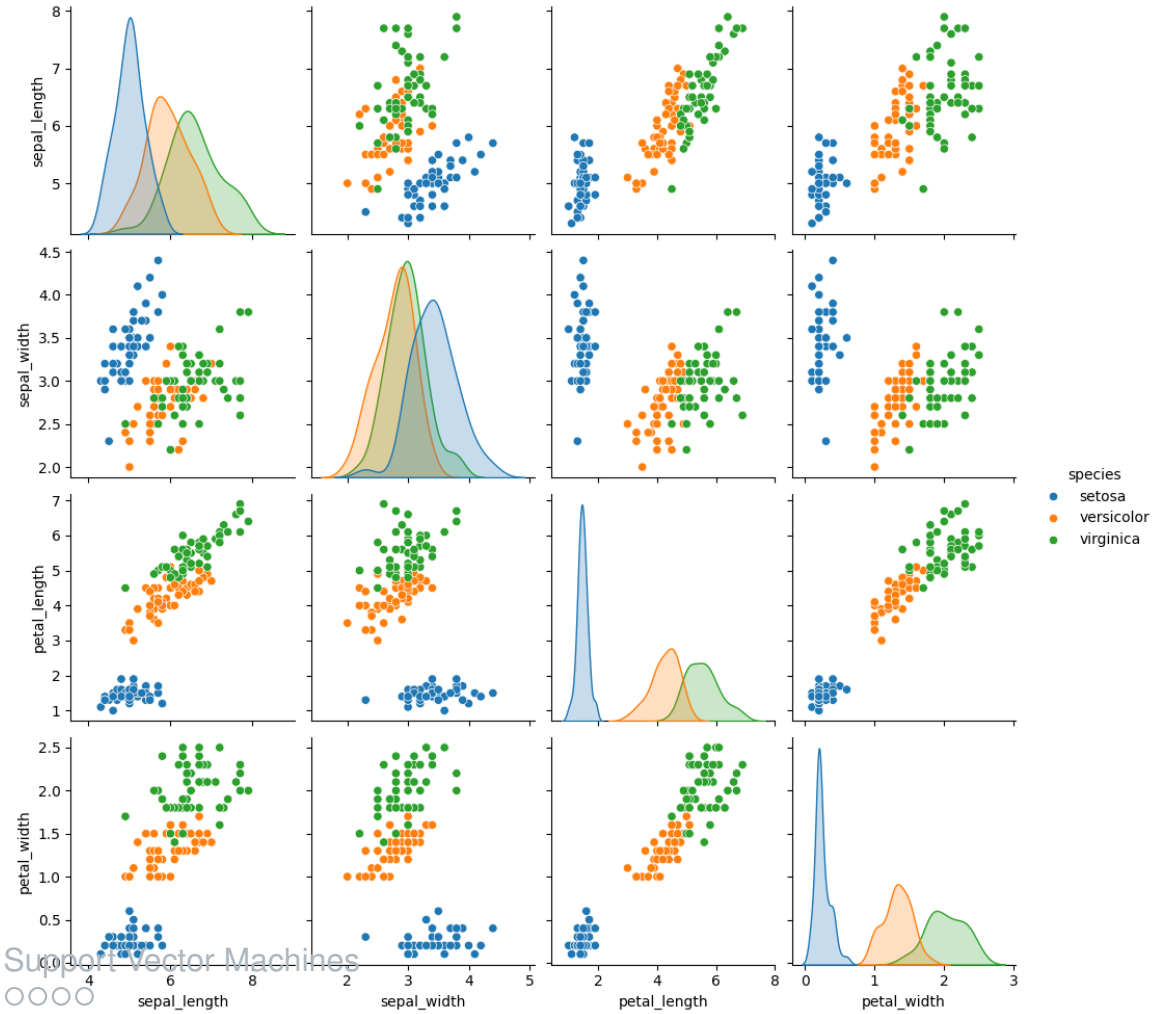
`pandas.DataFrame.dropna()`

- Remove missing values (NaNs)
- `axis=0`: rows containing missing values are removed, `axis=1`: columns are removed

Pair Plot

`sns.pairplot(DataFrame, hue="labels")`

- Easy way to look at obvious correlations between different features



Scaling Input Data

Scikit-learn `StandardScaler`

- Scale data to a mean value of $\mu = 0$ and standard deviation of $\sigma = 1$

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

- Create a StandardScaler instance:
  `scaler: StandardScaler = StandardScaler()`

- Compute the mean and standard deviation for each column for the scaling in the next step:
  `scaler.fit(training_data)`

- Actual standardization:
  `training_data_scaled = scaler.transform(training_data)`

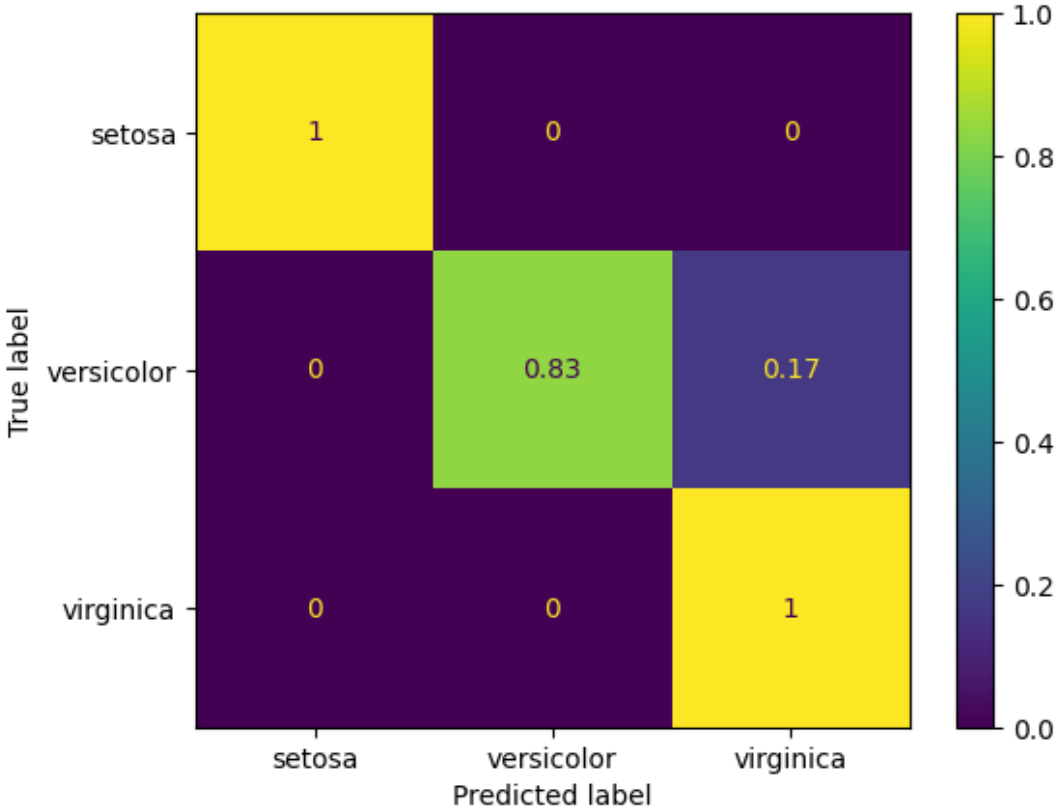
Logistic Regression

Scikit-learn `LogisticRegression` class

- Create a model
  ```
  model:  LogisticRegression = LogisticRegression()
  ```
- Fit the model to the training set
  ```
  model.fit(training_data, training_labels)
  ```
- Make predictions
  ```
  model.predict(test_data)
  ```
- Test the predictions, for example with the accuracy score
  ```
  metrics.accuracy_score(test_labels, predicted_labels)
  ```

Confusion Matrix

```
confusion_matrix(y_true, y_predicted)
```



Support Vector Machine

Scikit-learn `SVC` class

- Create a model with the RBF kernel, regularization parameter `C=2` and scaling factor `gamma=2`
  ```
  model:  SVC = SVC(kernel="rbf", C=2, gamma=2)
  ```

- Training, predicting and testing works like for logistic regression

Literature

- Logistic Regression: Deep Learning, Goodfellow, Bengio and Courville 2016
- Support Vector Machines: Deep Learning, Goodfellow, Bengio and Courville 2016 and StatQuest by Josh Starmer and The Elements of Statistical Learning, Hastie, Tibshirani, Friedman